

PHISHGUARD

Academic Project Documentation

Intelligent Email Phishing Analyzer | Think Before You Click

Document Information	Details
Project Domain	Cybersecurity, phishing detection, full-stack web application, explainable risk analysis
Prepared For	Academic project review and source-code submission
Document Version	Professional documentation edition
Repository Requirement	All project documents and source code are prepared for GitHub upload.

Project Abstract

PhishGuard was designed for a common everyday security problem: users often receive emails or links that look legitimate but may lead to credential theft, fraud, or malware. The project gives users a safer way to inspect suspicious content before taking action. It combines secure parsing, guarded URL checking, explainable risk scoring, report generation, feedback collection, and administrator governance in one working full-stack application.

Index & Document Navigation

How This Document Is Organized

This documentation is organized as a complete academic project report. It starts with the project idea and planning work, then moves through research, requirements, design, implementation, testing, and final reporting. The index is written for a lecturer, reviewer, or developer who needs to move through the project quickly without losing the overall story.

Table of Contents

The table of contents below is generated from the document headings and updated with Word page numbers during final verification.

Index & Document Navigation.....	2
Table of Contents.....	2
Document Map	4
Detailed Section Index	4
List of Figures	5
List of Key Tables.....	6
Reader Path.....	6
1. Project Planning & Management	7
1.1 Project Proposal	7
1.2 Project Plan	8
1.3 Task Assignment & Roles	9
1.4 Risk Assessment & Mitigation Plan.....	10
1.5 KPIs.....	10
2. Literature Review	12
2.1 Feedback & Evaluation.....	12
2.2 Suggested Improvements	12
2.3 Final Grading Criteria	13
3. Requirements Gathering.....	14
3.1 Stakeholder Analysis	14
3.2 User Stories & Use Cases	15
3.3 Functional Requirements.....	16
3.4 Non-Functional Requirements.....	16
4. System Analysis & Design	17

4.1 Problem Statement & Objectives	17
4.2 Use Case Diagram & Descriptions	17
4.3 Functional & Non-Functional Requirements	17
4.4 Software Architecture	18
4.5 Database Design & Data Modeling	19
4.6 Data Flow & System Behavior	21
4.7 UI/UX Design & Prototyping	24
4.8 System Deployment & Integration	25
4.9 Additional Deliverables	26
5. Implementation: Source Code & Execution	28
5.1 Source Code	28
5.2 Version Control & Collaboration	28
5.3 Deployment & Execution	29
6. Testing & Quality Assurance	30
6.1 Test Cases & Test Plan	30
6.2 Automated Testing	30
6.3 Bug Reports	31
7. Final Presentation & Reports	32
7.1 User Manual	32
7.2 Technical Documentation	32
7.3 Project Presentation	32
7.4 Video Walkthrough	33
Appendix A. Technical Reference	34
A.1 Hybrid Scoring	34
A.2 Environment Variables	34
A.3 References	35

Document Map

Index Table 1. High-level document map.

Part	Section	Page	What To Find There
Front Matter	Index & Document Navigation	p. 2	Navigation tables, figure list, reader paths, and the formal table of contents.
1	Project Planning & Management	p. 7	Project purpose, scope, responsibilities, risk controls, milestones, resources, and success measures.
2	Literature Review	p. 12	Research themes that influenced the solution, plus evaluation notes and improvement opportunities.
3	Requirements Gathering	p. 14	Stakeholders, user stories, use cases, functional requirements, and quality expectations.
4	System Analysis & Design	p. 17	Problem analysis, architecture, database model, workflows, UI/UX approach, deployment design, and API coverage.
5	Implementation: Source Code & Execution	p. 28	Code organization, standards, security controls, collaboration process, configuration, and execution guide.
6	Testing & Quality Assurance	p. 30	Manual and automated validation, browser smoke checks, expected outcomes, and resolved issues.
7	Final Presentation & Reports	p. 32	End-user guidance, technical summary, presentation structure, and video walkthrough plan.
Appendix	Technical Reference	p. 34	Scoring logic, classification bands, environment variables, and supporting references.

Detailed Section Index

Index Table 2. Detailed section index.

Section	Subsections	Page	Academic Deliverables
1	1.1 Proposal; 1.2 Project Plan; 1.3 Roles; 1.4 Risks; 1.5 KPIs	p. 7	Planning package with the project story, roadmap, responsibility matrix, risk register, and success metrics.
2	2.1 Feedback & Evaluation; 2.2 Improvements; 2.3 Grading Criteria	p. 12	Research context, assessment notes, planned improvements, and a clear evaluation rubric.
3	3.1 Stakeholders; 3.2 User Stories & Use Cases; 3.3 Functional Requirements; 3.4 Non-Functional Requirements	p. 14	Requirements package that explains user needs, scenarios, system features, and quality constraints.
4	4.1 Objectives; 4.2 Use Case; 4.3 Requirements; 4.4	p. 17	System analysis and design package with

	Architecture; 4.5 Database; 4.6 Behavior; 4.7 UI/UX; 4.8 Deployment; 4.9 Additional Deliverables		diagrams, schemas, API summary, testing strategy, and deployment strategy.
5	5.1 Source Code; 5.2 Version Control; 5.3 Deployment & Execution	p. 28	Implementation documentation covering standards, modularity, security handling, collaboration, and run instructions.
6	6.1 Test Plan; 6.2 Automated Testing; 6.3 Bug Reports	p. 30	Quality assurance package with expected outcomes, automation coverage, and issue-resolution history.
7	7.1 User Manual; 7.2 Technical Documentation; 7.3 Presentation; 7.4 Video Walkthrough	p. 32	Final reporting package for users, reviewers, and presentation delivery.
Appendix	A.1 Hybrid Scoring; A.2 Environment Variables; A.3 References	p. 34	Technical reference material for scoring, configuration, and standards.

List of Figures

Index Table 3. List of figures.

Figure	Title	Page	Purpose
1	Project Phase Roadmap	p. 8	Shows the academic project workflow by phase.
2	Use Case Diagram	p. 15	Shows user and administrator interactions.
3	System Architecture	p. 18	Shows the layered full-stack design.
4	Database ERD	p. 19	Shows the main database entities and relationships.
5	Data Flow Diagram	p. 21	Shows analysis, storage, model, and URL probe data flow.
6	Sequence Diagram	p. 22	Shows URL safety-check interaction flow.
7	Activity Diagram	p. 22	Shows the end-to-end analysis workflow.
8	State Diagram	p. 23	Shows analysis, feedback, and model governance states.
9	Class Diagram	p. 23	Shows the main domain model classes.
10	UI Wireframes	p. 24	Shows representative interface screens.
11	Deployment Diagram	p. 25	Shows runtime nodes and infrastructure mapping.
12	Component Diagram	p. 26	Shows high-level software modules and dependencies.

List of Key Tables

Index Table 4. List of key table groups.

Table Group	Page	Content Covered
Planning tables	p. 7	Project plan, milestones, resource allocation, role assignment, risk mitigation, and KPIs.
Review and evaluation tables	p. 12	Literature themes, final grading criteria, and improvement areas.
Requirements tables	p. 14	Stakeholders, user stories, functional requirements, and non-functional requirements.
Design tables	p. 17	Database schema, logical/physical schema, diagram descriptions, UI/UX decisions, technology stack, and API summary.
Implementation tables	p. 28	Source code quality, version control practices, README content, configuration, and execution guide.
Testing tables	p. 30	Test cases, automated testing coverage, and bug reports.
Final report tables	p. 32	Presentation outline, scoring reference, classification bands, and environment variables.

Reader Path

Index Table 5. Suggested reader paths.

Reader Goal	Recommended Path
Quick academic review	Read the cover, project abstract, index, Sections 1, 4, 6, and 7.
Technical review	Focus on Sections 4 and 5, then Appendix A.
Testing review	Read Sections 6 and 4.9, then check the bug report table.
Presentation preparation	Use Sections 7.3 and 7.4, supported by the figures listed above.

1. Project Planning & Management

This section explains how PhishGuard is managed as a real software project. It defines the purpose of the system, the boundaries of the work, the responsibilities of the team, the risks that must be controlled, and the indicators used to judge whether the project is successful.

1.1 Project Proposal

This subsection introduces the problem, the solution idea, and the practical boundaries of the project.

PhishGuard is a practical cybersecurity project built around a simple idea: people should be able to check suspicious messages and links before they trust them. Instead of showing raw technical output, the platform turns email, URL, sender, header, attachment, and model evidence into a clear risk score, explanation, and recommended action.

Element	Academic Description
Project Overview	A web-based phishing analyzer with registration, login, dashboard, analysis tools, report export, history, feedback, settings, and an administrator area.
Primary Objective	Help users make safer decisions by explaining why an email or URL appears safe, suspicious, or dangerous.
Technical Objective	Deliver a working full-stack application using Next.js, FastAPI, SQLAlchemy, PostgreSQL-ready storage, report generation, and a controlled machine-learning workflow.
Scope	The project covers email analysis, URL safety checking, sender/header review, file metadata checks, user history, reports, feedback, admin authorization, model governance, health monitoring, and Docker-based execution.
Boundaries	The system does not execute attachments or run unsafe website scripts. It uses guarded metadata checks so the analysis process remains controlled and safe.

1.2 Project Plan

The roadmap connects each academic phase to practical deliverables and measurable acceptance criteria.

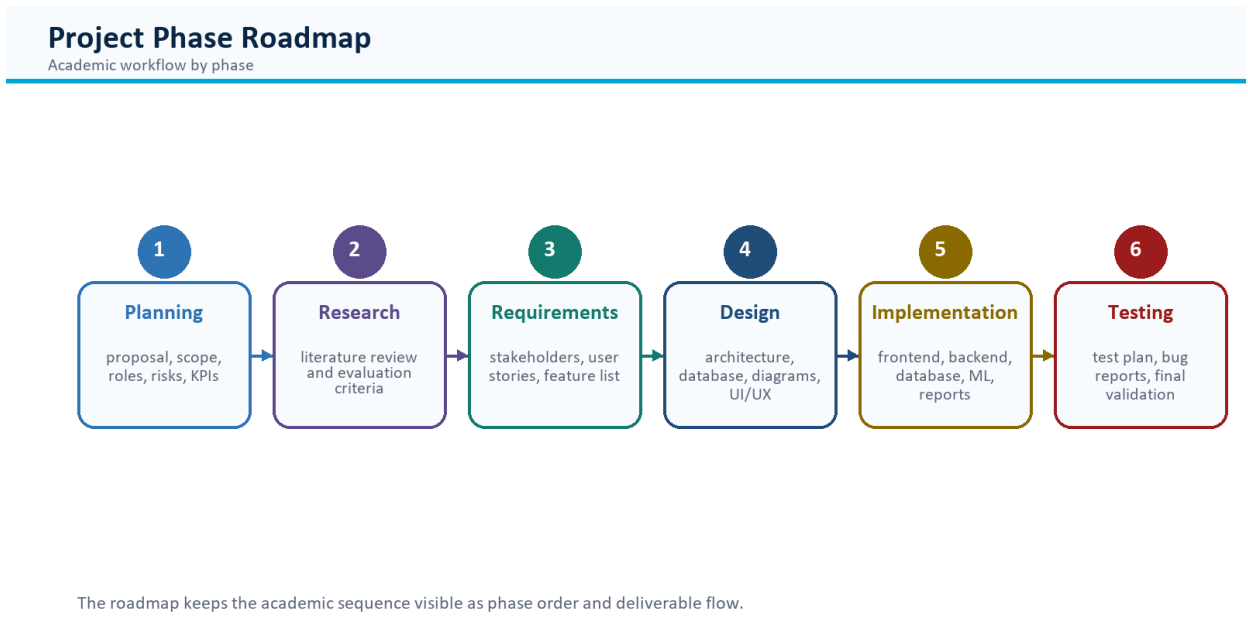


Figure 1. Phase roadmap showing the academic project workflow.

Table 1. Phase-based academic project plan.

Project Phase	Purpose	Key Deliverables
Project Planning & Management	Define the project direction, scope, responsibilities, success measures, and risk controls.	Proposal, project plan, role matrix, risk plan, KPIs.
Literature Review	Study phishing detection, email authentication, URL analysis, ML classification, and secure web design.	Research summary, evaluation notes, improvements, grading criteria.
Requirements Gathering	Identify stakeholders, system needs, user workflows, and measurable quality attributes.	Stakeholder analysis, user stories, use cases, functional and non-functional requirements.
System Analysis & Design	Translate requirements into diagrams, data models, UI/UX decisions, and architecture.	Use case diagram, ERD, DFD, sequence, activity, state, class, deployment, and component diagrams.
Implementation: Source Code & Execution	Build the full-stack application and prepare it for local or containerized execution.	Frontend, backend, database schema, analyzer modules, reports, admin controls, README.
Testing & Final Reports	Validate the application, document defects	Test plan, automated tests, bug

	and fixes, and prepare final delivery material.	reports, user manual, technical documentation, presentation outline.
--	---	--

Milestone	Deliverable	Acceptance Measure
M1	Proposal, scope, risk plan, KPIs	Project idea and success measures are clearly approved.
M2	Literature review	Relevant phishing, URL, ML, email authentication, and web security topics are summarized.
M3	Requirements	Stakeholders, user stories, functional requirements, and quality requirements are documented.
M4	System design	Use case, ERD, DFD, sequence, activity, state, class, component, and deployment diagrams are included.
M5	Implementation	Frontend, backend, database, reports, URL checker, and admin authorization work together.
M6	Testing and final report	Test cases, bug reports, user manual, technical documentation, and presentation outline are complete.

Resource	Allocation
Development tools	Visual Studio Code, Git/GitHub, Python, Node.js, Docker.
Frontend stack	Next.js, React, TypeScript, Tailwind CSS, Axios, Recharts, Lucide icons.
Backend stack	FastAPI, Pydantic, SQLAlchemy, JWT, Argon2, ReportLab, pytest.
Data/ML stack	scikit-learn, pandas, NumPy, joblib, verified datasets, versioned model artifacts.
Human resources	Project lead, backend developer, frontend developer, security/ML lead, QA/documentation owner.

1.3 Task Assignment & Roles

Responsibilities are separated so planning, coding, security, testing, and documentation all have clear ownership.

Role	Main Responsibilities	Outputs
Project Manager	Controls scope, schedule, milestones, GitHub uploads, and final delivery.	Plan, timeline, risk register, progress tracking.
Frontend Developer	Builds user interface, analyzer pages, dashboard, reports, settings, feedback, and admin screens.	Next.js pages, reusable components, UI tests.

Backend Developer	Implements APIs, database models, authentication, report generation, and business services.	FastAPI routes, SQLAlchemy schema, report endpoints.
Security/ML Engineer	Defines phishing indicators, URL probe safety, hybrid scoring, model training, evaluation, and activation.	Analyzers, model pipeline, model governance rules.
QA & Documentation Lead	Designs test cases, verifies flows, records bug reports, writes user/technical documentation.	Test plan, bug log, final DOCX, presentation outline.

1.4 Risk Assessment & Mitigation Plan

The risk register focuses on issues that could affect safety, accuracy, scope, or delivery quality.

Risk	Impact	Mitigation
Unsafe URL probing could contact private/internal systems.	High	Block non-public IP ranges, use DNS checks, support only HTTP/HTTPS, limit redirects, and avoid JavaScript execution.
ML model may be unavailable or inaccurate.	Medium	Use transparent rule-based scoring when no verified active model exists; train only with approved data.
False positives or false negatives.	High	Show explainable evidence, confidence, user feedback, admin review, and model rollback.
Unauthorized admin access.	High	Apply role-based access control, admin-only dependencies, and audit logs.
Sensitive user data exposure.	High	Use privacy mode, data export/delete, hashed passwords, JWT expiration, and no unnecessary file storage.
Schedule pressure.	Medium	Prioritize core workflows before optional integrations and maintain milestone-based tracking.

1.5 KPIs

The KPIs define how the project will be judged as a working system, not only as a design document.

KPI	Target	Measurement
Analysis response time	Common email or URL analysis completes within a few seconds.	Manual timing and API logs.
System uptime	Application remains available during review/testing sessions.	Health endpoints and Docker health checks.
User adoption readiness	A new user can register, analyze, export, and submit feedback without developer help.	User acceptance testing.
Security coverage	URL, sender, headers, language,	Report content review.

	attachment, and ML/rule evidence are shown.	
Test pass rate	Automated backend/frontend checks pass before submission.	pytest, Vitest, lint, build.
Admin governance	Admin can manage users, feedback, model versions, health, and audit logs.	Admin workflow tests.

2. Literature Review

Phishing detection is not solved by one technique alone. Attackers may hide behind convincing language, look-alike domains, mismatched senders, shortened links, or attachments that appear harmless. For that reason, PhishGuard follows a hybrid approach: it combines rule-based security evidence, guarded URL inspection, sender/header checks, language signals, and optional machine-learning support. The result is designed to be understandable, not only technically correct.

Review Area	Relevance to PhishGuard
Email authentication	SPF, DKIM, and DMARC concepts help explain whether a sender is likely to be authorized or suspicious.
URL safety analysis	Redirects, TLS status, reachability, content type, and unusual URL structure can reveal risk before a user clicks.
Machine learning	Text features and verified training samples can support classification, but the application still needs clear evidence and fallback behavior.
Secure parsing	Emails, headers, and attachments must be inspected without running scripts, loading remote content, or executing files.
Human feedback	Feedback is useful only when reviewed carefully; approved feedback can improve model training while reducing poisoning risk.
Web application security	Authentication, authorization, validation, rate limiting, security headers, and audit logs make the platform safe to operate.

2.1 Feedback & Evaluation

This evaluation view highlights what a reviewer should verify in the finished system.

- The project should be evaluated as a working full-stack application with real authentication, analysis, reporting, feedback, and administration workflows.
- The lecturer should verify registration, login, URL checking, email analysis, report export, feedback, and admin authorization.
- Evaluation should inspect whether analysis results are explainable and whether admin-only operations are properly protected.
- The final submission should include source code, documentation, test cases, reports, and presentation material in GitHub.

2.2 Suggested Improvements

These improvements extend the project after the core academic requirements are complete.

- Integrate optional reputation APIs such as Google Safe Browsing or VirusTotal when keys are available.
- Add live DNS-based SPF, DKIM, and DMARC verification beyond parsing header evidence.

- Improve .msg parsing with a hardened parser and more malicious file samples.
- Move rate limiting to Redis for scalable multi-instance deployment.
- Add a richer CI/CD pipeline with automatic Docker build, test, and deployment stages.

2.3 Final Grading Criteria

The grading model links marks to visible evidence in the repository, application, and final presentation.

Evaluation Category	Weight	Evidence
Documentation	20%	Proposal, plan, literature review, requirements, diagrams, API/database documentation, user manual.
Implementation	35%	Working frontend, backend, database, analyzer, URL checker, reports, admin authorization, ML workflow.
Testing	20%	Test plan, automated tests, bug reports, validation screenshots or logs.
Security and Quality	15%	RBAC, safe URL probe, secure parsing, validation, password hashing, audit logs.
Presentation	10%	Slides/PDF, live walkthrough, optional video, clear explanation of challenges and outcomes.

3. Requirements Gathering

The requirements were written from the viewpoint of the people who will use, review, maintain, or govern the platform. This keeps the system focused on real workflows: checking a suspicious URL, reviewing an email, exporting evidence, giving feedback, and managing the system as an administrator.

3.1 Stakeholder Analysis

Stakeholders are grouped by what they need from the system and what the application provides.

Stakeholder	Needs	System Response
General user	Check suspicious emails, links, files, and headers.	Analyzer workflow with clear report and recommended action.
Administrator	Full authorization and control over users, feedback, models, health, and audit logs.	Role-based admin panel and protected API endpoints.
Security reviewer	Evidence, transparency, and risk reasoning.	Indicators, component scores, URL details, sender/header analysis.
Lecturer/evaluator	Academic documentation, diagrams, tests, and working project execution.	Structured DOCX, source code, README, tests, and presentation outline.
Developer/maintainer	Clear setup and maintainable code.	Modular backend/frontend, typed schemas, Docker Compose, README.

3.2 User Stories & Use Cases

The user stories describe the main workflows that must feel reliable during review and presentation.

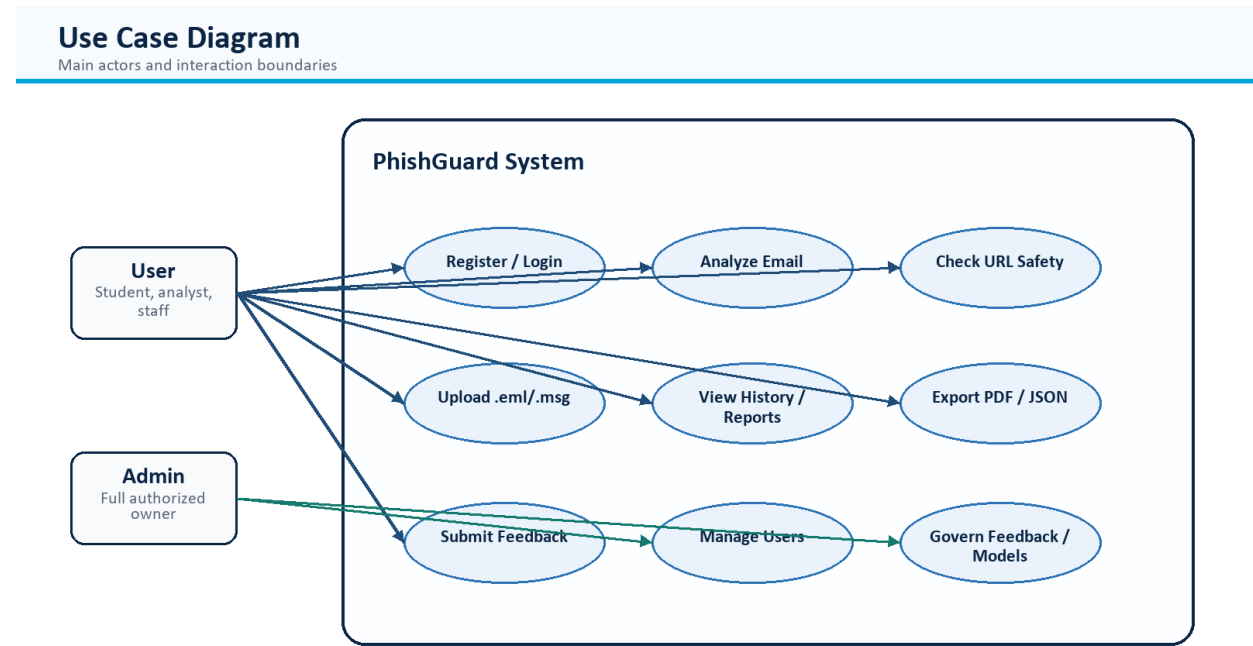


Figure 2. Use case diagram with user and administrator actors.

ID	User Story	Acceptance Criteria
US-01	As a visitor, I want to register so that my analysis history is protected.	Strong password validation, duplicate email prevention, authenticated session returned.
US-02	As a user, I want to paste a URL so that the system checks if it is safe.	URL is normalized, live metadata is collected safely, and verdict is displayed.
US-03	As a user, I want to analyze an email so that I know whether it is phishing.	Report shows score, classification, confidence, evidence, and recommended action.
US-04	As a user, I want to export reports so that I can share evidence.	PDF/JSON/CSV export options are available where appropriate.
US-05	As an admin, I want full authorized access to manage system resources.	Admin role can manage users, feedback, models, health, and audit logs.

3.3 Functional Requirements

Functional requirements translate the requested capabilities into clear build targets.

ID	Functional Requirement	Priority
FR-01	Support registration, login, logout, token refresh, profile update, and password change.	High
FR-02	Analyze pasted email content including sender, subject, body, headers, and embedded URLs.	High
FR-03	Analyze URLs and display safety verdict, reachability, final URL, TLS status, redirect behavior, and evidence.	High
FR-04	Support raw header analysis and .eml/.msg file upload within configured safety limits.	High
FR-05	Store analysis history for authenticated users unless privacy mode is enabled.	High
FR-06	Export reports and user data in reviewer-friendly formats.	Medium
FR-07	Allow users to submit feedback on analysis results.	Medium
FR-08	Allow admins to manage users, feedback, model versions, system health, and audit logs.	High

3.4 Non-Functional Requirements

Quality requirements define how the system should behave under normal use, risk, and failure conditions.

Category	Requirement
Performance	Use bounded URL probes, timeouts, and efficient analysis to keep reports responsive.
Security	Use Argon2 password hashing, JWT, RBAC, validation, rate limits, secure headers, and safe parsing.
Usability	Show readable results, not raw JSON; provide clear summaries and recommended actions.
Reliability	Continue deterministic analysis when ML model is unavailable and label model status clearly.
Maintainability	Keep modular code, typed schemas, reusable UI components, tests, and README instructions.
Privacy	Support privacy mode, data export, analysis deletion, and account deletion.

4. System Analysis & Design

This section explains how the requirements become a working architecture. It connects the problem statement to the system components, database structure, process flows, interface decisions, deployment model, and API surface.

4.1 Problem Statement & Objectives

The problem statement explains why the system is needed and what successful protection looks like.

Users frequently receive emails and links that imitate trusted brands, create urgency, hide destinations, or request credentials. The main objective is to provide an explainable system that checks the content safely and tells the user what risk exists and what action to take.

- Identify phishing indicators from message text, URLs, sender identity, headers, attachments, and language.
- Safely check URLs without executing remote JavaScript or opening private/internal targets.
- Provide a professional report with classification, score, evidence, and recommended action.
- Enable administrator governance over users, feedback, model versions, health, and audit records.

4.2 Use Case Diagram & Descriptions

The use case model separates normal user workflows from administrator governance workflows.

The user actor performs analysis and reporting workflows. The administrator actor has additional full authorization to govern accounts, feedback, model lifecycle, system health, and audit logs.

4.3 Functional & Non-Functional Requirements

The design preserves the required features while respecting security, privacy, and reliability constraints.

Functional requirements define what the system does, including analysis, authentication, export, feedback, and administration. Non-functional requirements define how well the system performs, including security, reliability, privacy, and usability.

4.4 Software Architecture

The architecture uses clear frontend, backend, data, analysis, and reporting layers.

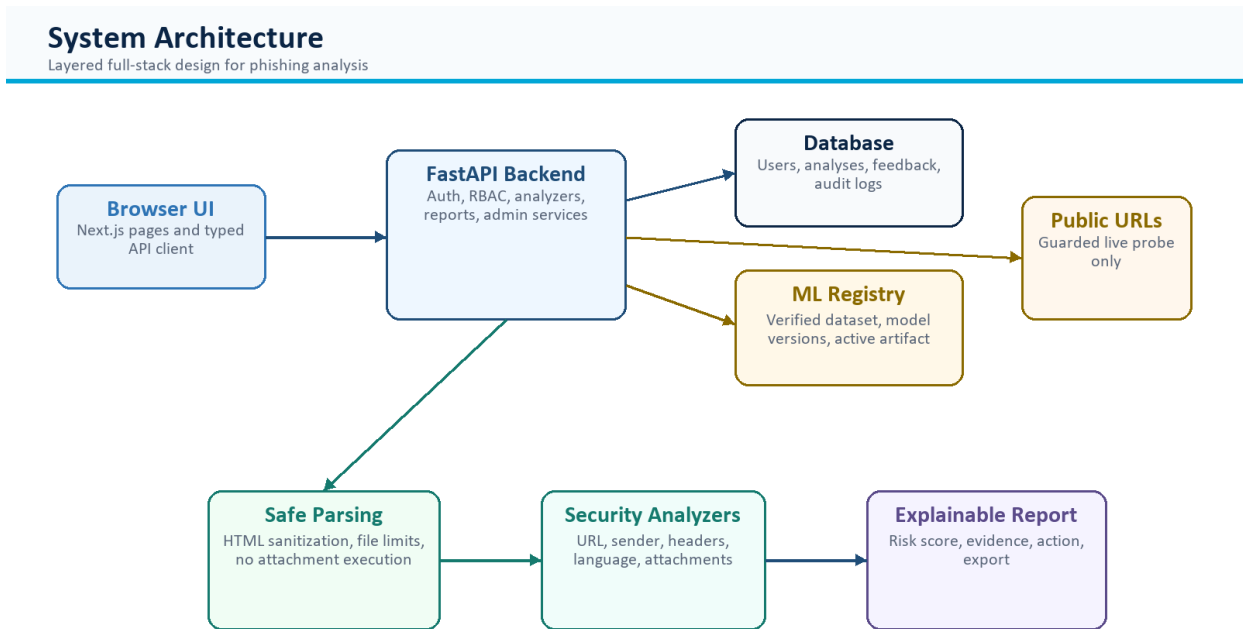


Figure 3. Layered architecture of the PhishGuard platform.

PhishGuard uses a layered architecture. The Next.js frontend provides the user interface. The FastAPI backend exposes typed APIs, applies authentication and authorization, orchestrates analyzers, stores results, and generates reports. SQLAlchemy models persist data, while the ML registry controls versioned model artifacts.

4.5 Database Design & Data Modeling

The data model keeps identity, reports, evidence, feedback, model governance, and audit activity separated.

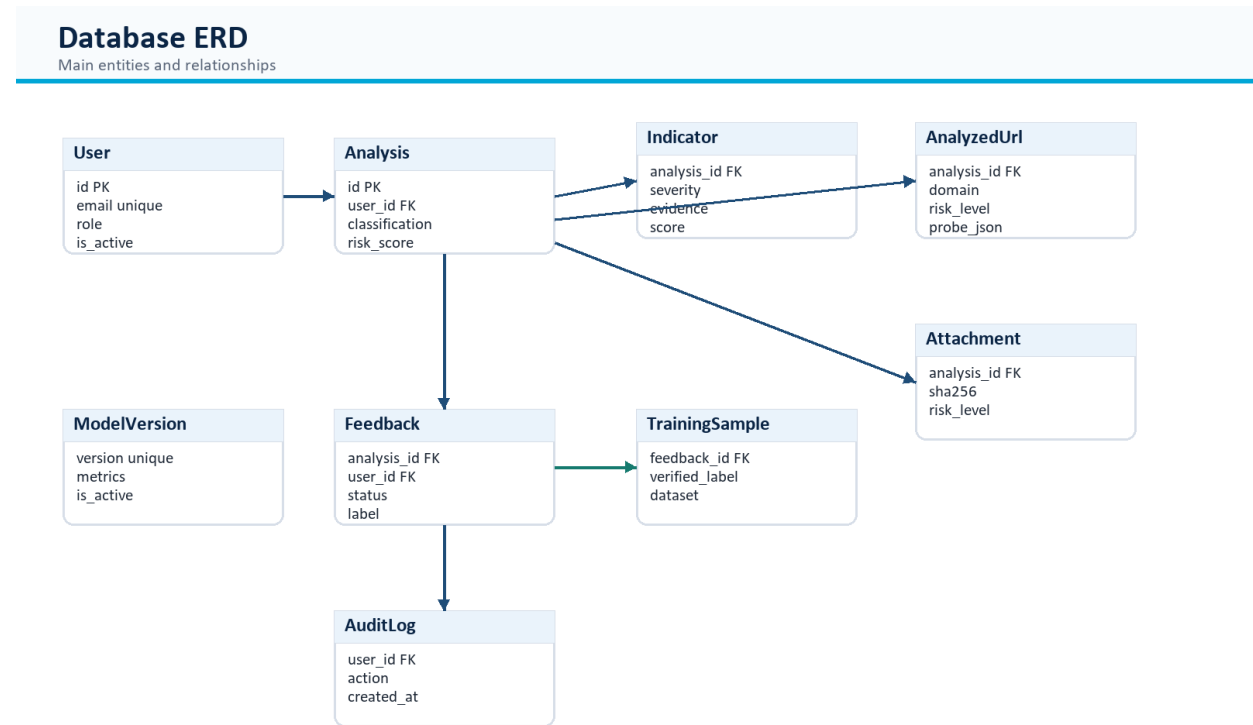


Figure 4. Entity Relationship Diagram for core database tables.

Entity	Purpose	Key Attributes
User	Stores registered accounts and authorization role.	id, full_name, email, password_hash, role, is_active
Analysis	Stores each report and final risk decision.	id, user_id, classification, risk_score, confidence, raw_result_json
EmailIndicator	Stores explainable evidence items.	analysis_id, indicator_type, severity, evidence, score_contribution
AnalyzedUrl	Stores URL findings and live probe metadata.	original_url, domain, risk_score, safety_verdict, findings_json
Attachment	Stores attachment metadata without execution.	filename, mime_type, extension, sha256, risk_level
Feedback	Stores user feedback awaiting admin review.	feedback_type, suggested_label, status, reviewed_by
TrainingSample	Stores approved samples for model training.	email_data_json, verified_label, dataset_version
ModelVersion	Stores model artifact metadata and active	version, model_path, metrics_json, is_active

	state.	
AuditLog	Stores security and admin activity.	user_id, action, entity_type, entity_id, created_at

The schema is normalized so that users, analyses, indicators, URLs, attachments, feedback, training samples, model versions, and audit logs are stored in separate tables with foreign-key relationships. This avoids repeated data and supports reporting, deletion, and governance.

Logical Schema and Physical Schema: The logical model defines the system entities and relationships, while the physical schema maps those entities to concrete SQLAlchemy/PostgreSQL-ready tables, keys, indexes, and JSON evidence fields.

Schema View	Design Details
Logical Schema	Entities are separated by responsibility: identity, analysis reports, evidence indicators, URL findings, attachment metadata, feedback, training data, model registry, audit records, and settings.
Physical Schema	Tables use UUID-style string primary keys, indexed lookup fields, foreign keys with cascade behavior for user-owned analysis data, and JSON text fields for flexible analyzer evidence.
Normalization	Repeated evidence is stored in child tables instead of duplicating it inside the analysis row; raw_result_json keeps a complete report snapshot for export and replay.

4.6 Data Flow & System Behavior

The behavior diagrams show how inputs move through validation, analysis, storage, reporting, and governance.

Data Flow Diagram

How data moves through analysis, storage, and governance

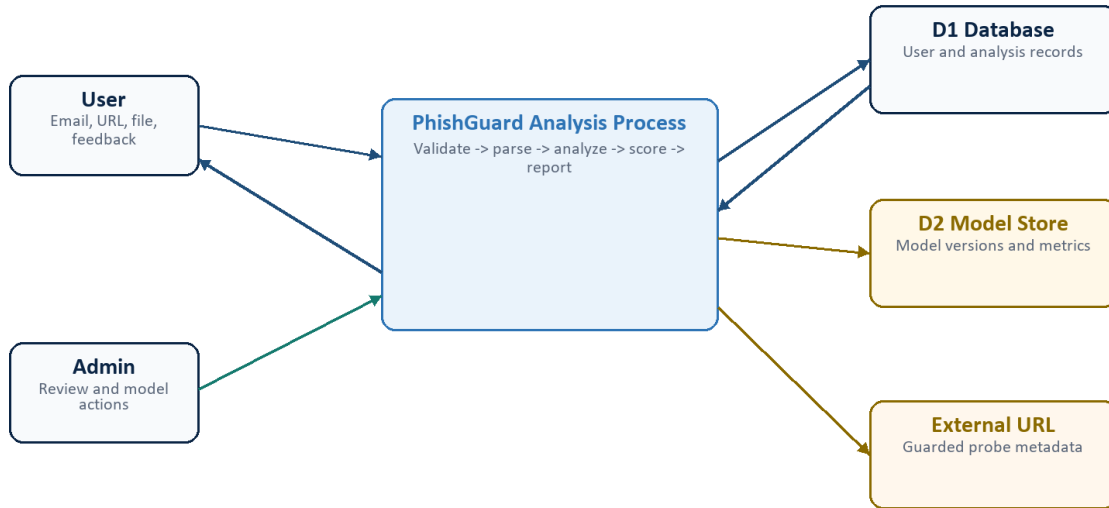


Figure 5. Data Flow Diagram for analysis, storage, and model interaction.

Sequence Diagram

URL safety check interaction flow

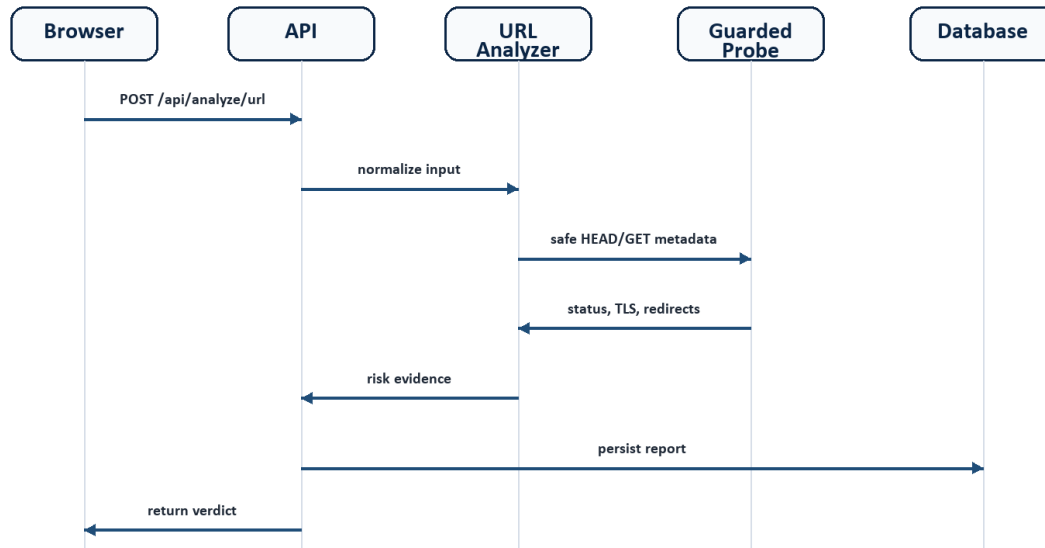


Figure 6. Sequence Diagram for URL safety checking.

Activity Diagram

End-to-end analysis workflow

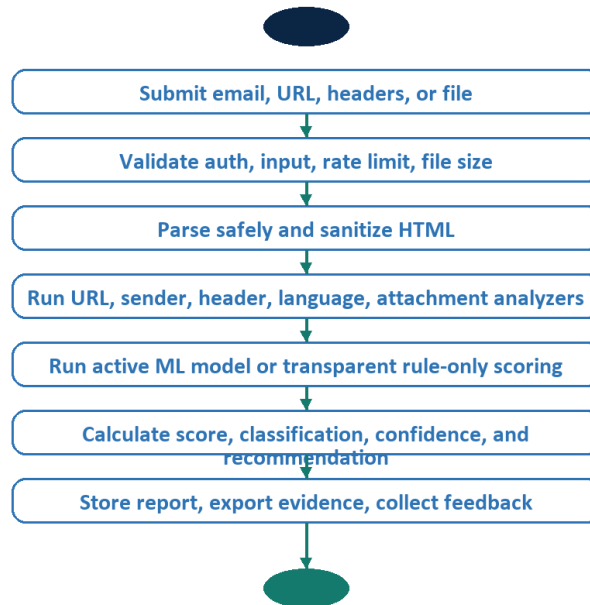


Figure 7. Activity Diagram for the primary analysis workflow.

State Diagram

Analysis, feedback, and model governance states

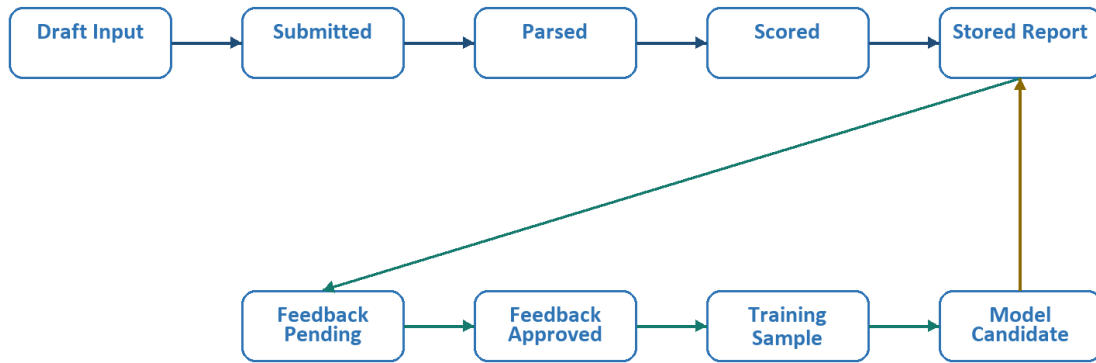


Figure 8. State Diagram for analysis, feedback, and model governance.

Class Diagram

Domain model structure and relationships

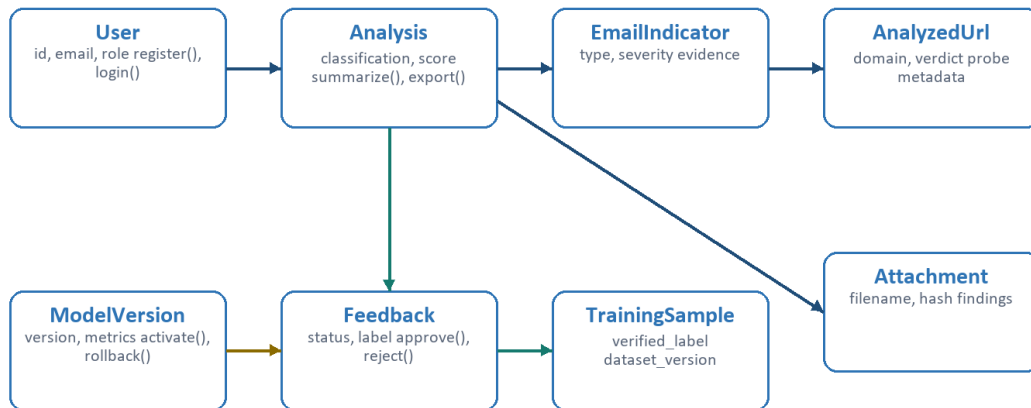


Figure 9. Class Diagram for the core domain model.

Diagram Type	Description
DFD	Shows how user/admin inputs move through authentication, parsing, analysis, scoring, storage, model registry, and URL probing.
Sequence Diagram	Represents the ordered interaction between browser, API, analyzer, URL probe, and database during URL analysis.
Activity Diagram	Shows workflow from submission to validation, parsing, analysis, scoring, reporting, and feedback.
State Diagram	Shows state transitions from draft input to submitted, parsed, scored, stored report, feedback, training sample, and model candidate.
Class Diagram	Shows domain classes such as User, Analysis, Indicator, AnalyzedUrl, Attachment, Feedback, TrainingSample, ModelVersion, and AuditLog.

4.7 UI/UX Design & Prototyping

The interface is designed for quick security review: clear inputs, readable results, and no raw technical clutter.

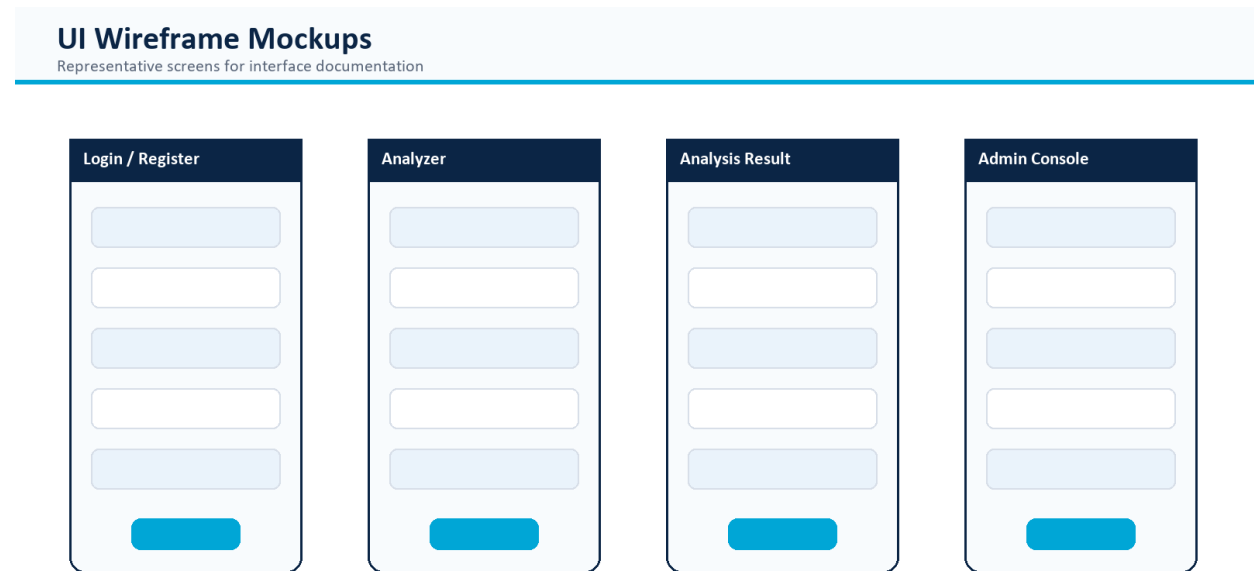


Figure 10. Wireframes for major PhishGuard screens.

UI Area	Design Decision
Login/Register	Simple form layout with clear validation and no default sample credentials.
Analyzer	Tabbed workflow for email, URL, headers, and file upload, with privacy mode and a clear

	submit action.
Analysis Result	Risk gauge, executive summary, recommended action, URL details, sender/header findings, and export controls.
Admin	Structured panels for users, feedback, model versions, system health, and audit logs.
Accessibility	Readable contrast, clear labels, accessible risk meter, keyboard-friendly controls, and no raw JSON for normal users.
Visual identity	Dark security dashboard with cyan/blue accents, severity colors, compact cards, and readable tables.

4.8 System Deployment & Integration

The deployment model supports local execution, containerized review, and future hosted deployment.

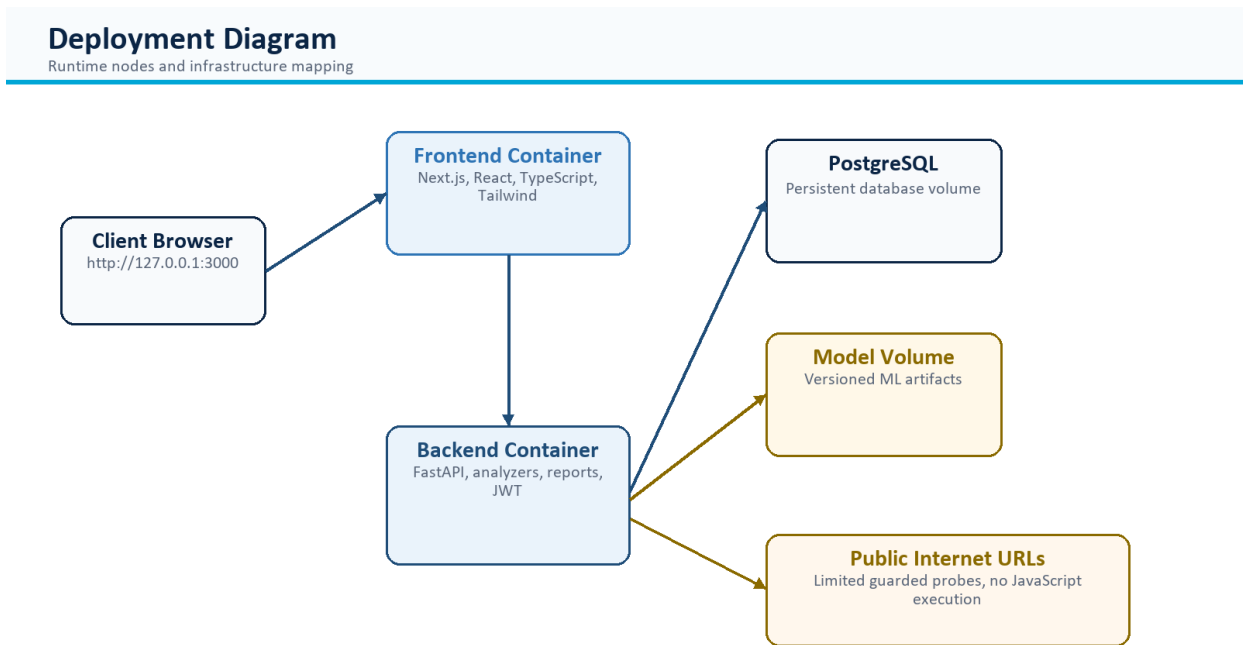


Figure 11. Deployment Diagram showing runtime nodes.

Component Diagram

Major software modules and dependencies

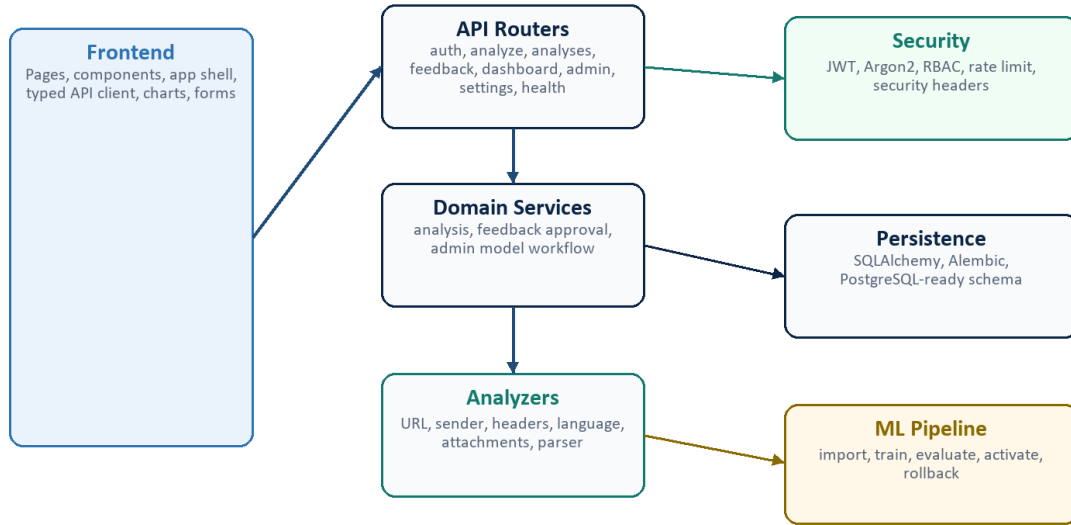


Figure 12. Component Diagram showing high-level module dependencies.

Layer	Technology
Frontend	Next.js, React, TypeScript, Tailwind CSS, Axios, Recharts, Lucide icons.
Backend	FastAPI, Pydantic, SQLAlchemy, Alembic, JWT, Argon2, ReportLab.
Database	SQLite for local development and PostgreSQL through Docker Compose.
Machine Learning	scikit-learn, pandas, NumPy, joblib, TF-IDF, logistic regression, model registry.
Deployment	Docker Compose with frontend, backend, database, health checks, and model artifact volume.

4.9 Additional Deliverables

This subsection summarizes the API surface, validation approach, and deployment strategy that support the source code.

API Area	Representative Endpoints
Authentication	POST /api/auth/register, POST /api/auth/login, POST /api/auth/refresh, GET /api/auth/me
Analysis	POST /api/analyze/email, POST /api/analyze/url, POST /api/analyze/headers, POST /api/analyze/file
Reports/History	GET /api/analyses, GET /api/analyses/{id}, GET /api/analyses/{id}/report, GET /api/analyses/export.csv

Feedback	POST /api/analyses/{id}/feedback, GET /api/feedback/my-feedback
Admin	GET /api/admin/users, PATCH /api/admin/users/{id}, GET /api/admin/feedback, POST /api/admin/models/train, GET /api/admin/system-health
Settings/Health	PATCH /api/settings/profile, GET /api/settings/export, DELETE /api/settings/account, GET /health, GET /ready

- Testing and validation include unit tests, integration tests, user acceptance tests, linting, build checks, and browser smoke checks.
- Deployment strategy uses local development during implementation and Docker Compose for reproducible project review.
- Optional production deployment can use managed PostgreSQL, HTTPS, environment secrets, CI/CD, and persistent model storage.

5. Implementation: Source Code & Execution

The implementation is organized as a full-stack project rather than a isolated screen. The frontend, backend, database layer, analyzers, reports, authentication, administration, and model workflow are separated so the system can be tested, maintained, and extended.

5.1 Source Code

The source code is structured around clear modules, consistent standards, secure handling, and reusable services.

Code Area	Implementation Quality
Structured code	Backend route modules, service modules, analyzer modules, schemas, models, and ML scripts are separated by responsibility.
Commented code	Complex behavior such as URL probing, scoring, model governance, and parsing is organized into readable functions.
Coding standards	Typed Python, Pydantic validation, SQLAlchemy mappings, TypeScript frontend, reusable UI components, and consistent naming.
Modularity	URL, sender, header, language, attachment, parser, ML, feedback, and admin services can evolve independently.
Security	Argon2 passwords, JWT, RBAC, rate limiting, validation, safe parsing, private URL blocking, and audit logging.
Error handling	HTTP exceptions, readable UI error messages, model-unavailable transparency, and upload validation.

5.2 Version Control & Collaboration

The repository workflow supports controlled changes, readable history, and repeatable quality checks.

Item	Professional Practice
Repository	Upload source code, README, docs, tests, diagrams, final DOCX, and presentation files to GitHub.
Branching Strategy	Use main for stable releases, develop for integration, and feature branches for auth, analyzer, admin, ML, reports, and docs.
Commit History	Use meaningful messages such as feat: add guarded URL probe, fix: render API errors, test: admin access control.
Pull Requests	Include scope, screenshots, tests run, database impact, security notes, and reviewer checklist.
CI/CD	Recommended pipeline: install dependencies, run backend tests, run frontend lint/tests,

	build frontend, build Docker images, deploy.
--	--

5.3 Deployment & Execution

Execution guidance explains how a reviewer or developer can run the project locally or through containers.

README Item	Content to Include
Installation steps	Backend virtual environment, pip install, frontend npm install, environment variables, database setup.
System requirements	Python, Node.js, npm, Docker, browser, PostgreSQL or SQLite for development.
Configuration	DATABASE_URL, JWT_SECRET_KEY, CORS_ORIGINS, MODEL_STORAGE_PATH, URL probe settings, optional API keys.
Execution guide	Run backend on port 8000 and frontend on port 3000, or use docker compose up --build.
API documentation	FastAPI documentation available through backend docs endpoint during local execution.
Executable/deployment link	For a web project, provide the deployed web app URL or Docker Compose instructions instead of a desktop executable.

6. Testing & Quality Assurance

Testing focuses on the flows that prove the application is real and usable: account creation, login, URL analysis, email analysis, report display, export, feedback, access control, owner-safe history, and administrator actions. The test plan combines expected manual outcomes with automated checks.

6.1 Test Cases & Test Plan

The test cases describe what should happen when users and administrators perform key actions.

Test Case	Scenario	Expected Outcome
TC-01	Register with a strong password.	Account is created and authenticated session is returned.
TC-02	Register with a weak password.	System rejects request with readable validation error.
TC-03	Login with invalid credentials.	System returns unauthorized error without exposing sensitive details.
TC-04	Analyze a safe URL.	System normalizes URL, performs guarded check, and displays safe or low-risk evidence.
TC-05	Analyze uppercase/freeform URL input.	System extracts URL case-insensitively and analyzes it.
TC-06	Analyze private/internal URL.	System blocks live probe and records blocked reason.
TC-07	Analyze phishing-style email.	Report shows high-risk evidence and protective recommended action.
TC-08	Upload unsupported file.	System rejects the upload with a clear error.
TC-09	Normal user opens admin endpoint.	System denies access.
TC-10	Admin manages feedback/model.	Admin can review feedback and govern model versions.

6.2 Automated Testing

Automated checks currently include 20 passing backend pytest tests, 7 passing frontend unit tests, a passing frontend lint run, a successful production build, and browser smoke coverage for login, analysis, result, and admin flows. One browser case remains intentionally skipped in the suite.

Test Type	Coverage
Backend unit/integration tests	20 passing pytest tests covering authentication, analysis persistence, uploads, reports, feedback, admin access, model status, and owner-safe

	history.
Frontend tests	7 passing unit tests covering risk gauge formatting and accessible meter rendering.
Lint/build checks	ESLint and the Next.js production build pass successfully.
Browser smoke tests	5 browser smoke checks passed; 1 case is intentionally skipped in the suite.

6.3 Bug Reports

The bug log records problems found during implementation and the corrective action taken.

Bug / Issue	Resolution
Registration was not working correctly.	Fixed registration integration, validation, and authentication flow.
React runtime error when rendering API object.	Converted API errors into readable strings before display.
Uppercase URL input did not check correctly.	Updated URL extraction and normalization to be case-insensitive.
System health showed raw JSON.	Redesigned admin health information into readable status fields.
Admin account required full authorization.	Implemented admin role access for users, feedback, models, health, and audit logs.
Placeholder behavior reduced trust in the system.	Removed sample-only users, synthetic dashboard stats, and hidden synthetic model assumptions.
Feedback from regular users was not clear in admin review.	Admin Feedback Queue now shows submitter identity, pending state, manual refresh, and periodic refresh.
Admin review displayed the current admin as the analyst for user-created records.	Analysis list responses now include original owner fields, and History/CSV exports display the real analyst.
Correct feedback needed a consistent dataset label.	Correct-result feedback is normalized from the analysis classification before approval into the verified dataset.

7. Final Presentation & Reports

The final reporting section gives the material needed to explain the finished project to users, reviewers, and an academic panel. It includes end-user instructions, technical documentation notes, presentation structure, and an optional video walkthrough plan.

7.1 User Manual

The user manual explains the everyday workflow in simple operational steps.

1. Open the application in the browser.
2. Register or log in with a valid account.
3. Open the Analyzer page and select Email, URL, Headers, or File.
4. Paste suspicious content or upload a supported .eml/.msg file.
5. Submit the analysis and review the risk score, evidence, summary, and recommended action.
6. Export the report as needed.
7. Submit feedback if the result is correct, false positive, false negative, or unclear.
8. Administrators can open the Admin page to manage users, feedback, models, health, and audit logs.

7.2 Technical Documentation

Technical documentation links the application behavior to its architecture, database schema, API surface, and tests.

- System architecture is documented through layered architecture and deployment diagrams.
- Database schema is documented through ERD and entity table descriptions.
- API documentation is summarized in this report and available from the FastAPI docs endpoint during execution.
- Testing documentation includes test cases, automated tests, and bug resolution records.

7.3 Project Presentation

The presentation should tell the project story clearly: problem, design choices, implementation, validation, and outcomes.

Slide	Suggested Content
1	Title, project purpose, and active model status.
2	Presentation roadmap and review path.
3	Phishing problem and why evidence matters.
4	Analysis pipeline from raw input to verdict.
5	Controlled workflow from login to report.
6	Architecture and technology stack.

7	URL safety checking and guarded probing.
8	NLP and machine learning contribution.
9	Explainable result page and exports.
10	Admin controls, feedback review, and user governance.
11	Database traceability and stored evidence.
12	Latest testing and quality-assurance evidence.
13	Readiness, limitations, and future improvements.

7.4 Video Walkthrough

The optional video walkthrough should show the system working from login to analysis and administration.

9. Introduce PhishGuard and its purpose.
10. Show registration/login.
11. Analyze a URL and explain the safety verdict.
12. Analyze a suspicious email and explain the evidence.
13. Export a report.
14. Submit feedback.
15. Log in as admin and show user, feedback, model, health, and audit controls.
16. Close with testing results and future enhancements.

Appendix A. Technical Reference

The appendix keeps detailed reference material separate from the main explanation. It is useful for reviewers who want to inspect scoring logic, configuration, and the external standards that influenced the system.

A.1 Hybrid Scoring

The scoring reference explains how risk evidence contributes to the final classification.

Component	Weight	Evidence
Machine Learning	45%	Verified model prediction and top factors.
URLs	20%	URL structure, redirects, TLS, content type, reachability, blocked reason.
Sender	15%	Display name, sender domain, reply-to mismatch, impersonation signals.
Authentication	10%	SPF/DKIM/DMARC-style header evidence.
Attachments	5%	File extension, MIME type, size, hash, executable indicators.
Language	5%	Urgency, credential requests, threats, payment/account language.

Score	Classification	Severity
0-19	Safe	Informational
20-39	Low Risk	Low
40-59	Suspicious	Medium
60-79	Phishing	High
80-100	Critical Threat	Critical

A.2 Environment Variables

Configuration values control database access, security, model storage, CORS, and optional reputation integrations.

Variable	Purpose
DATABASE_URL	Database connection string.
JWT_SECRET_KEY	Secret key used to sign tokens.
CORS_ORIGINS	Allowed frontend origins.

MODEL_STORAGE_PATH	Location for versioned model artifacts.
ENABLE_URL_LIVE_PROBE	Controls guarded URL probing.
URL_PROBE_TIMEOUT_SECONDS	Maximum wait time for URL metadata checks.
SAFE_BROWSING_API_KEY	Optional reputation API integration key.
VIRUSTOTAL_API_KEY	Optional reputation API integration key.

A.3 References

The references list the standards, security guidance, and project artifacts used to support the report.

- RFC 9989, RFC 9990, and RFC 9991 for current DMARC concepts.
- RFC 7208 for SPF sender authorization concepts.
- RFC 6376 for DKIM signature concepts.
- OWASP ASVS for secure web application verification guidance.
- PhishGuard local README, architecture document, backend schemas, domain models, and tests.